==========

AI-ASSIGEMENT-13.4

==========

NAME:-NIPUN.I **H.T.NO:-2303A52208**

==========

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions Week 7

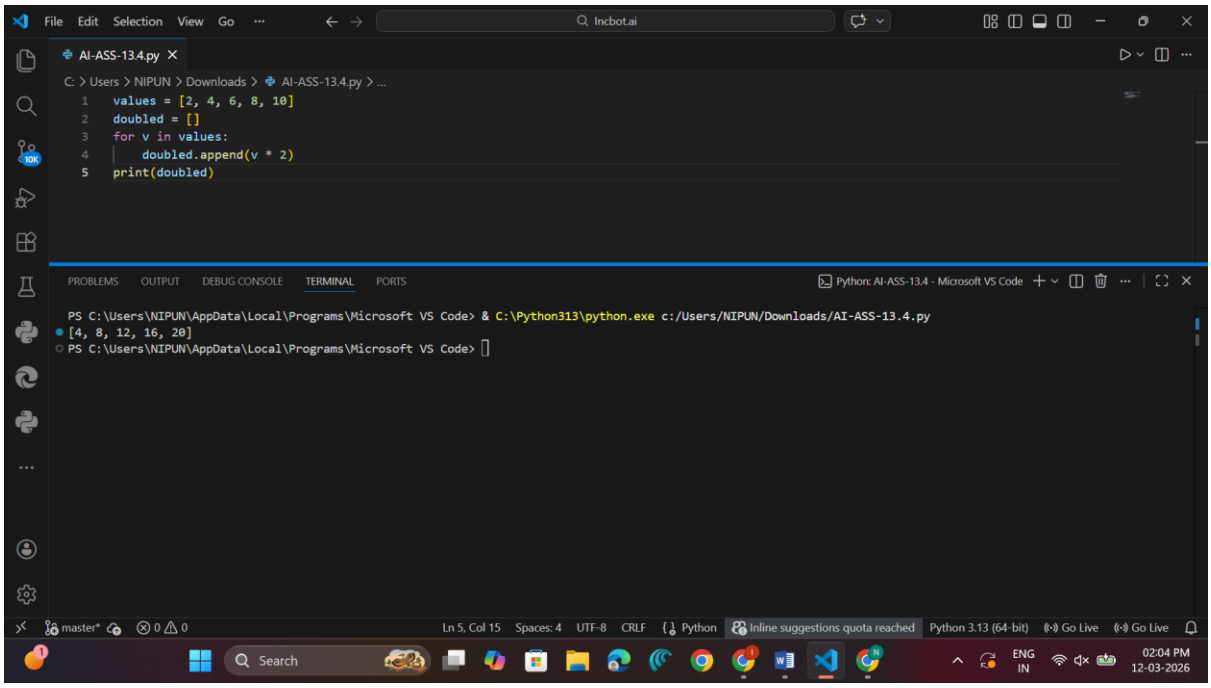
==========

Task 1 – Refactoring Data Transformation

Prompt:

I have a Python list of numbers and a for-loop that doubles each value and appends it to a new list. How can I refactor this to be more Python and efficient using a list comprehension?

Code & Output:



Observation:

The original code used a loop and `append()` method to add elements to a list. After refactoring, **list comprehension** was used, which makes the code shorter, cleaner, and more readable while producing the same output.

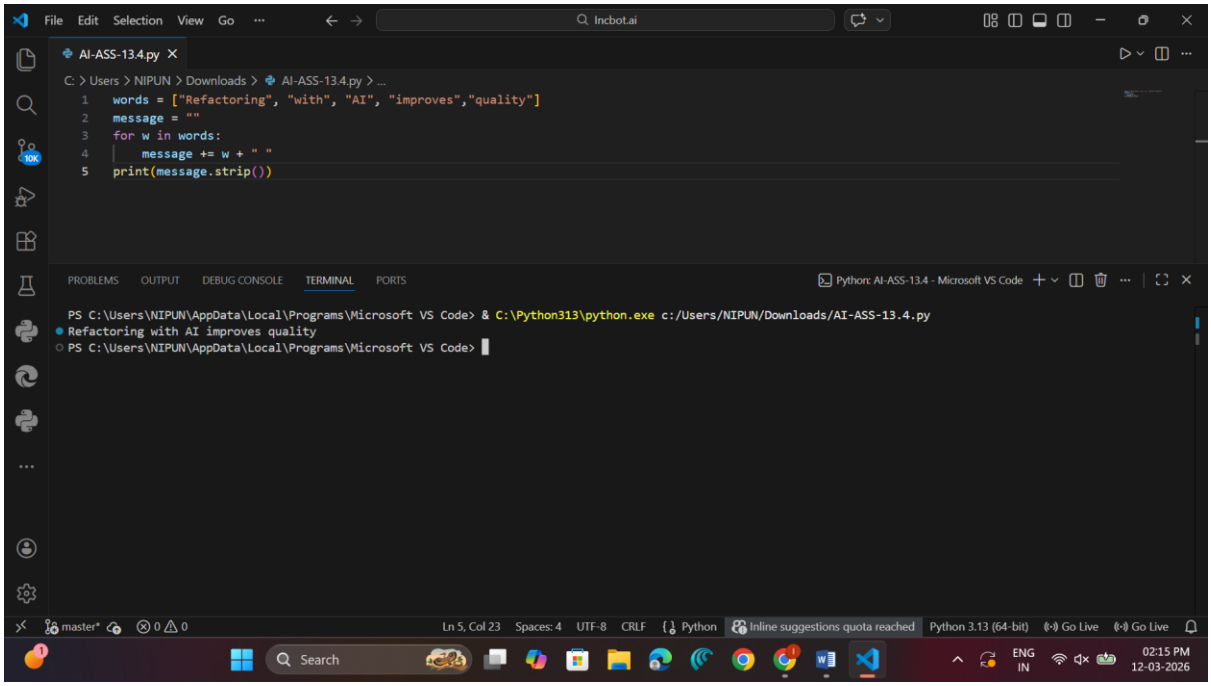
==========

Task 2 – Improving Text Processing Code

Prompt:

Suggest a more efficient and readable way to construct a sentence from a list of words instead of repeatedly concatenating strings inside a loop.

Code & Output:



Observation:

The legacy code used repeated string concatenation, which can be inefficient. The refactored version uses the **`join()` method**, which combines all words in the list with a space separator, making the code simpler and more efficient.

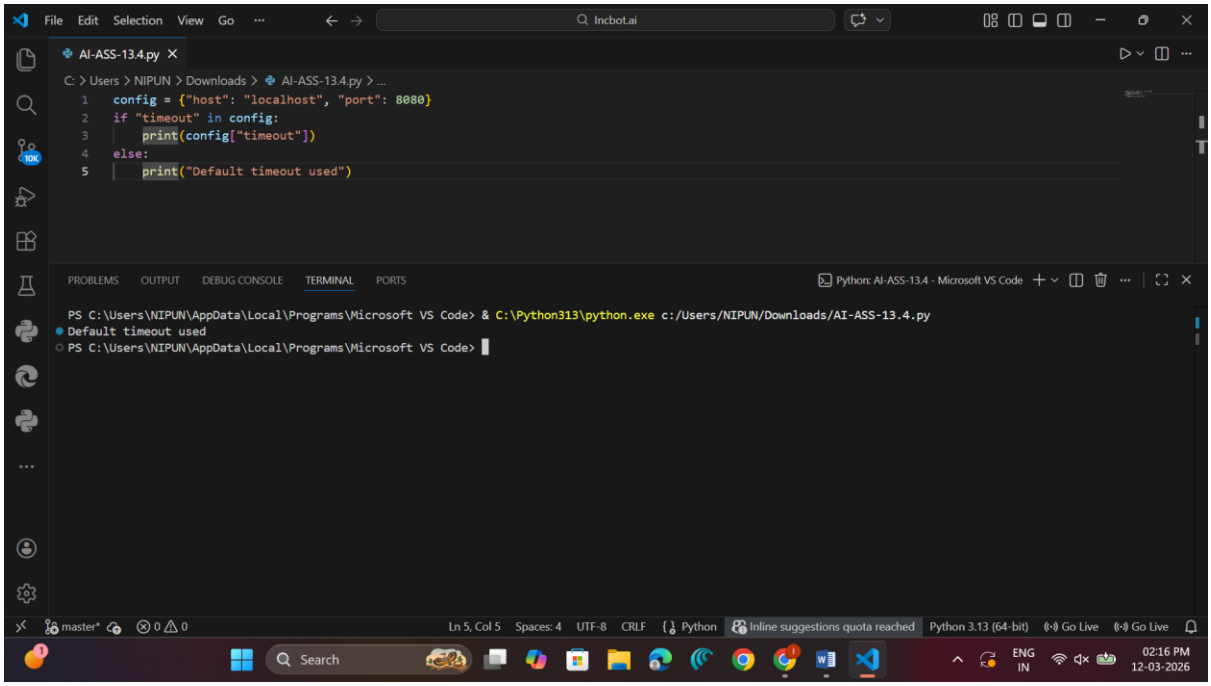
==========

Task 3 – Safer Access to Configuration Data

Prompt:

Refactor the code that manually checks for dictionary keys by using a safer dictionary access method in Python.

Code & Output:



Observation:

Instead of manually checking if a key exists in the dictionary, the **`get()` method** is used. This method safely retrieves the value if the key exists and returns a default value if the key is missing, making the code more concise and reliable.

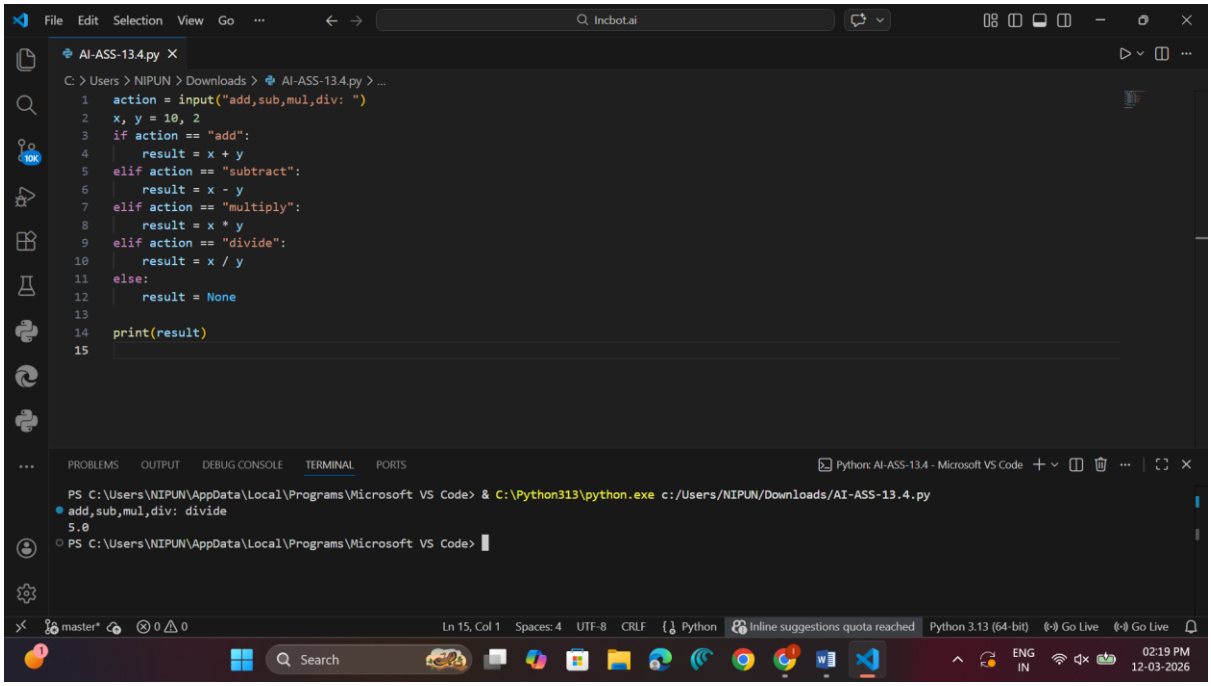
==========

Task 4 – Refactoring Conditional Logic

Prompt:

Replace multiple `if-elif` conditions used for different operations with a cleaner and more scalable mapping technique.

Code & Output:



Observation:

The original code used many `if-elif` statements to perform operations. The refactored code uses a **dictionary mapping approach**, which improves readability, reduces code length, and makes it easier to add new operations in the future.

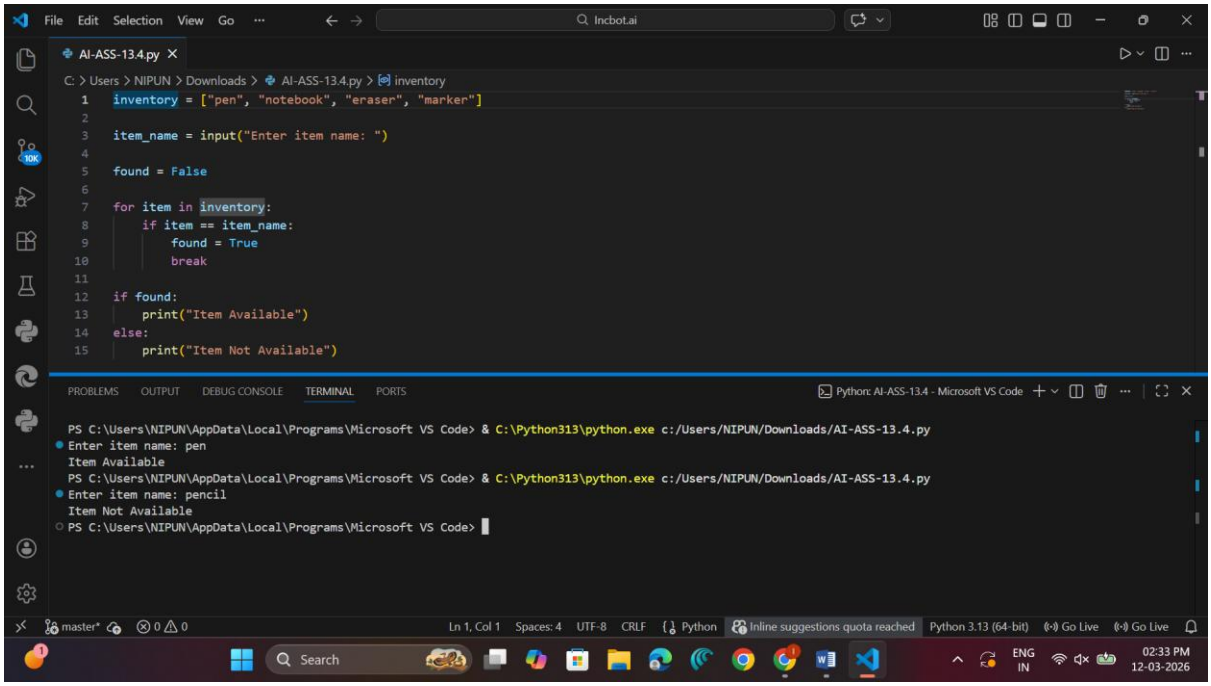
==========

Task 5 – Simplifying Search Logic

Prompt:

Refactor the manual loop used to search for an item in a list by using a more concise Pythonic approach.

Code & Output:



Observation:

The legacy code used a loop and a flag variable to check if an item existed in the list. The refactored code uses the **`in` operator**, which directly checks for membership in the list and makes the code shorter and easier to understand.